

第 54 讲 实验设计

Scaled Dot-Product Attention : Scale、Mask、Softmax、Value Aggregation 与 Backward

实验总目标

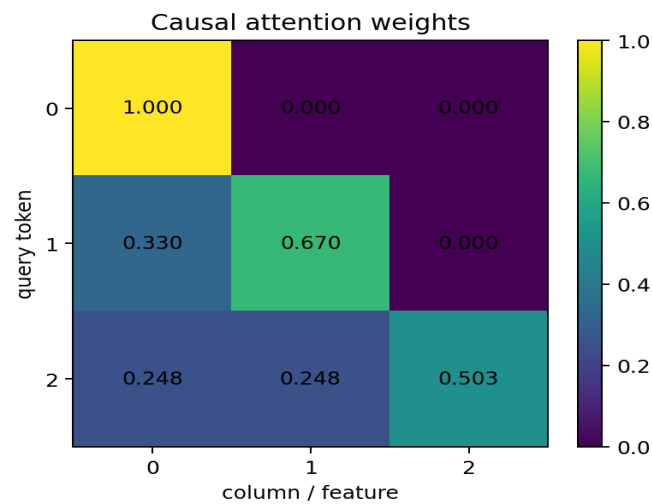
用极小矩阵把完整 Attention 逐步算出来，并验证 scaling、causal mask、PyTorch fused API、Backward 和 Batch/Head Shape。

实验原则

本实验不是性能 benchmark。所有数字刻意选得很小，以便手算与 PyTorch 对照。性能实现（如 FlashAttention）留到以后按需理解。

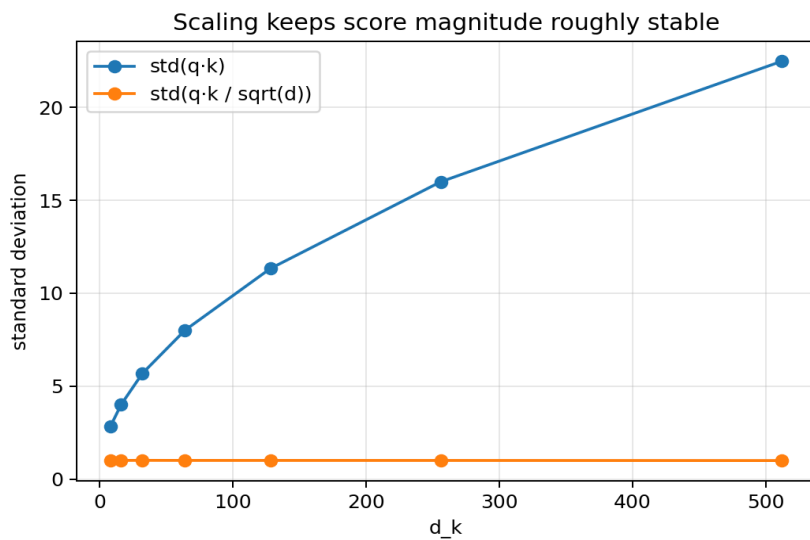
实验 1-2：完整 Attention 与 Causal Mask

步骤	验证
QK^T	3×3 score matrix
$/\text{sqrt}(d_k)$	score 尺度变小但排序不变
causal mask	未来 score $\rightarrow -\text{inf}$
softmax	未来权重严格 0, 行和=1
$\text{weights} @ V$	得到每个 Query 的上下文输出

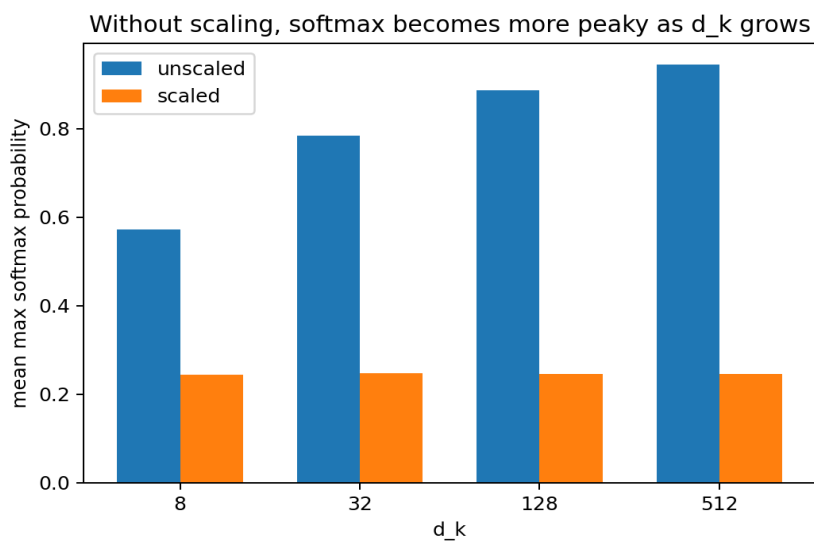


实验 3-4：为什么要 Scaling

随机生成 q/k ，每个分量服从标准正态。改变 d_k ，统计 Dot Product 的标准差。



再比较 16 个 Key 下 Softmax 最大概率：不缩放时 d_k 越大越尖；缩放后保持更稳定。



实验 5-7：框架对齐与错误 Mask 顺序

将手写 causal attention 与：

```
F.scaled_dot_product_attention(Q, K, V, is_causal=True, dropout_p=0.0)
```

做数值对照。然后故意“先 Softmax、再把未来权重置 0”，观察行和不再等于 1，证明 Mask 顺序的重要性。

验收

正确实现中 future weight max=0，manual 与 PyTorch 最大差异应仅为浮点误差。

实验 8-9：Backward 与 Shape

对 Q/K/V 设置 `requires_grad=True`，经过 attention 后构造标量 Loss，执行 backward，验证 Q/K/V 都收到非零 Gradient。

再构造：

Q,K,V: (B,H,T,d) = (2,4,5,8)

验证：

Scores: (2,4,5,5)

Output: (2,4,5,8)

结论边界

这些实验验证数学与计算图，不证明 Attention Weight 具备因果解释性，也不证明某种 Attention 变体一定提升任务性能。